FRD

FUNCTIONAL REQUIREMENTS DOCUMENT (FRD)

Customer Order Analytics ETL

- Document Version: 1.0
- Date: 2024-01-15
- Prepared By: Senior Business Analyst
- Project: Customer Order Analytics ETL
-

1. DOCUMENT OVERVIEW

1.1 Purpose

This Functional Requirements Document (FRD) defines the complete functional specifications for the Customer Order Analytics ETL pipeline. It translates the business requirements into implementation-ready functional requirements that will enable the technical team to design and develop an AWS Glue-based solution for integrating customer and order data.

1.2 Scope

This FRD covers:

- Data extraction from CSV files stored in S3
- Data quality validation and cleansing
- Customer and order data integration via inner join
- Derived field calculations and transformations
- Output generation in Parquet format
- Error handling and logging requirements
- Testing requirements for unit and integration testing

This FRD does NOT cover:

- Infrastructure provisioning or IAM policies
- Downstream analytics or reporting logic
- Real-time streaming or incremental processing
- Data archival or retention policies

1.3 Assumptions

1. Both source CSV files contain headers in the first row
2. CSV delimiter is comma (,) and character encoding is UTF-8

3. Source files are available at the specified S3 locations with consistent naming

4. AWS Glue execution environment has necessary read/write permissions configured

5. No schema evolution is expected during initial implementation phase

6. Processing will be batch-based with daily frequency

7. All configuration parameters will be externalized via YAML configuration file

8. Current timestamp for PROCESSED_DATE will be captured at job execution time

9. Proper case conversion follows standard title case rules (first letter uppercase, rest lowercase)

10. ORDER_COUNT calculation includes all orders for a customer, even if the same order appears multiple times after join

•

2. FUNCTIONAL REQUIREMENTS

FR-EXTRACT-001: Extract Customer Data from S3

• Title: Load Customer Master Data from CSV

• Description:

The system shall read customer master data from a CSV file stored in S3 and load it into a DataFrame for processing.

• Business Objective:

Enable access to customer master data as the foundation for customer-order analytics.

• Inputs:

• Source: s3://adif-sdlc/busi_req_doc/customers.csv

• Format: CSV with header row

• Schema:

• CID: Integer (Customer ID, Primary Key, NOT NULL)

• FNAME: String (First Name, NOT NULL)

• LNAME: String (Last Name, NOT NULL)

• Processing Rules / Functional Steps:

1. Connect to S3 location: s3://adif-sdlc/busi_req_doc/customers.csv

2. Read CSV file with header=true option

3. Infer or apply schema with columns: CID (Integer), FNAME (String), LNAME (String)

4. Load all records into a customer DataFrame

5. Log the count of records loaded

• Outputs:

- Customer DataFrame containing all records from source file
- Log entry with record count

- Preconditions:
- Source file exists at specified S3 location
- File is accessible with valid read permissions
- File follows expected CSV format with headers

- Postconditions:
- Customer data is loaded in memory as a DataFrame
- Record count is logged for audit trail

- Error / Validation Conditions:
- If file does not exist, log error and terminate job
- If file is empty, log warning and continue with empty DataFrame
- If schema cannot be parsed, log error and terminate job
- If file format is invalid (not CSV), log error and terminate job
-

FR-EXTRACT-002: Extract Order Data from S3

- Title: Load Order Transaction Data from CSV

- Description:

The system shall read order transaction data from a CSV file stored in S3 and load it into a DataFrame for processing.

- Business Objective:

Enable access to order transaction data for integration with customer information.

- Inputs:
- Source: s3://adif-sdlc/busi_req_doc/orders.csv
- Format: CSV with header row
- Schema:
- OID: Integer (Order ID, Primary Key, NOT NULL)
- CID: Integer (Customer ID, Foreign Key, NOT NULL)
- ONAME: String (Order Name/Description, NOT NULL)

- Processing Rules / Functional Steps:

1. Connect to S3 location: s3://adif-sdlc/busi_req_doc/orders.csv

2. Read CSV file with header=true option

3. Infer or apply schema with columns: OID (Integer), CID (Integer), ONAME (String)

4. Load all records into an order DataFrame

5. Log the count of records loaded

• Outputs:

• Order DataFrame containing all records from source file

• Log entry with record count

• Preconditions:

• Source file exists at specified S3 location

• File is accessible with valid read permissions

• File follows expected CSV format with headers

• Postconditions:

• Order data is loaded in memory as a DataFrame

• Record count is logged for audit trail

• Error / Validation Conditions:

• If file does not exist, log error and terminate job

• If file is empty, log warning and continue with empty DataFrame

• If schema cannot be parsed, log error and terminate job

• If file format is invalid (not CSV), log error and terminate job

•

FR-VALIDATE-001: Validate Customer Data Quality

• Title: Apply Data Quality Rules to Customer Records

• Description:

The system shall validate customer records against defined data quality rules and remove invalid or duplicate records.

• Business Objective:

Ensure only clean, valid customer data is used for analytics to maintain data integrity and reporting accuracy.

• Inputs:

• Customer DataFrame from FR-EXTRACT-001

• Processing Rules / Functional Steps:

1. Mandatory Field Validation:

• Check CID is NOT NULL

• Check FNAME is NOT NULL and not empty string

• Check LNAME is NOT NULL and not empty string

• Flag records failing any check as invalid

2. Data Type Validation:

• Verify CID is a positive integer (> 0)

• Flag records with CID <= 0 as invalid

3. Duplicate Removal:

• Identify duplicate records based on CID

• Keep first occurrence, remove subsequent duplicates

• Log count of duplicates removed

4. Error Logging:

• Write all invalid records to error log with rejection reason

• Include original record data and validation failure details

5. Continue Processing:

• Proceed with valid records only

• Do not terminate job on validation failures

• Outputs:

• Validated customer DataFrame containing only valid, unique records

• Error log with rejected records and reasons

• Validation summary metrics (total records, valid records, invalid records, duplicates removed)

• Preconditions:

• Customer DataFrame is loaded and available

• Postconditions:

• All records in customer DataFrame pass validation rules

• No duplicate CID values exist in customer DataFrame

• Invalid records are logged for review

• Error / Validation Conditions:

• CID is NULL → Reject record, log "CID is NULL"

• CID <= 0 → Reject record, log "CID must be positive integer"

• FNAME is NULL or empty → Reject record, log "FNAME is required"

• LNAME is NULL or empty → Reject record, log "LNAME is required"

• Duplicate CID → Remove duplicate, log "Duplicate CID removed"

•

FR-VALIDATE-002: Validate Order Data Quality

• Title: Apply Data Quality Rules to Order Records

• Description:

The system shall validate order records against defined data quality rules and remove invalid or duplicate records.

• Business Objective:

Ensure only clean, valid order data is used for analytics to maintain data integrity and reporting accuracy.

• Inputs:

• Order DataFrame from FR-EXTRACT-002

• Processing Rules / Functional Steps:

1. Mandatory Field Validation:

• Check OID is NOT NULL

• Check CID is NOT NULL

• Check ONAME is NOT NULL and not empty string

• Flag records failing any check as invalid

2. Data Type Validation:

• Verify OID is a positive integer (> 0)

• Verify CID is a positive integer (> 0)

• Flag records with OID <= 0 or CID <= 0 as invalid

3. Duplicate Removal:

• Identify duplicate records based on OID

• Keep first occurrence, remove subsequent duplicates

• Log count of duplicates removed

4. Error Logging:

• Write all invalid records to error log with rejection reason

• Include original record data and validation failure details

5. Continue Processing:

• Proceed with valid records only

• Do not terminate job on validation failures

• Outputs:

• Validated order DataFrame containing only valid, unique records

• Error log with rejected records and reasons

• Validation summary metrics (total records, valid records, invalid records, duplicates removed)

• Preconditions:

• Order DataFrame is loaded and available

• Postconditions:

• All records in order DataFrame pass validation rules

• No duplicate OID values exist in order DataFrame

• Invalid records are logged for review

• Error / Validation Conditions:

• OID is NULL → Reject record, log "OID is NULL"

• OID <= 0 → Reject record, log "OID must be positive integer"

• CID is NULL → Reject record, log "CID is NULL"

• CID <= 0 → Reject record, log "CID must be positive integer"

• ONAME is NULL or empty → Reject record, log "ONAME is required"

• Duplicate OID → Remove duplicate, log "Duplicate OID removed"

•

FR-TRANSFORM-001: Create Full Customer Name

• Title: Derive Full Name from First and Last Name

• Description:

The system shall create a new column FULL_NAME by concatenating FNAME and LNAME with proper case formatting.

• Business Objective:

Provide a single, standardized customer name field for reporting and analytics.

• Inputs:

• Validated customer DataFrame from FR-VALIDATE-001

• Columns: FNAME (String), LNAME (String)

• Processing Rules / Functional Steps:

1. Convert FNAME to proper case (title case): first letter uppercase, remaining lowercase

2. Convert LNAME to proper case (title case): first letter uppercase, remaining lowercase

3. Concatenate converted FNAME and LNAME with a single space separator

4. Store result in new column FULL_NAME

5. Preserve original FNAME and LNAME columns for join operation

• Outputs:

• Customer DataFrame with additional column FULL_NAME (String)

• Preconditions:

• Customer DataFrame contains valid FNAME and LNAME values

• FNAME and LNAME are not NULL or empty (validated in FR-VALIDATE-001)

• Postconditions:

• FULL_NAME column exists in customer DataFrame

• FULL_NAME follows format: "Firstname Lastname" in proper case

• Original FNAME and LNAME columns remain unchanged

• Error / Validation Conditions:

• If FNAME or LNAME contains only whitespace, trim before concatenation

• If proper case conversion fails, log warning and use original case

•

FR-TRANSFORM-002: Join Customer and Order Data

• Title: Integrate Customer and Order Information

• Description:

The system shall perform an inner join between customer and order data using CID as the join key to create an integrated customer-order dataset.

• Business Objective:

Create a unified view of customers and their orders for comprehensive analytics, including only customers who have placed orders.

• Inputs:

• Validated customer DataFrame from FR-TRANSFORM-001 (with FULL_NAME)

• Validated order DataFrame from FR-VALIDATE-002

• Processing Rules / Functional Steps:

1. Perform INNER JOIN between customer and order DataFrames

2. Join condition: customer.CID = order.CID

3. Select columns from result:

• CID (from customer)

• FULL_NAME (from customer)

• OID (from order)

• ONAME (from order)

4. Verify referential integrity: all orders in result have matching customer records

5. Log count of customers included and excluded from result

• Outputs:

• Joined DataFrame containing columns: CID, FULL_NAME, OID, ONAME

• Log entry with join statistics (customer count, order count, joined record count)

• Preconditions:

• Customer DataFrame is validated and contains FULL_NAME

• Order DataFrame is validated

• Both DataFrames contain CID column

• Postconditions:

• Every record in joined DataFrame has a valid customer and order

• No orphan orders (orders without customers) exist in result

• Customers without orders are excluded from result

• Error / Validation Conditions:

• If no matching records found, log warning and produce empty DataFrame

• If order CID does not exist in customer data, record is excluded (inner join behavior)

• Log count of orders excluded due to missing customer reference

•

FR-TRANSFORM-003: Calculate Order Count per Customer

• Title: Derive Order Count Metric

• Description:

The system shall calculate the total number of orders for each customer and add this as a new column ORDER_COUNT.

• Business Objective:

Provide order frequency metric to enable customer segmentation and purchase pattern analysis.

• Inputs:

• Joined DataFrame from FR-TRANSFORM-002 (columns: CID, FULL_NAME, OID, ONAME)

• Processing Rules / Functional Steps:

1. Group records by CID
2. Count the number of order records (OID) for each CID
3. Use window function partitioned by CID to calculate count
4. Add ORDER_COUNT column to each record
5. Ensure ORDER_COUNT value is consistent for all orders of the same customer

• Outputs:

• Enhanced DataFrame with additional column ORDER_COUNT (Integer)

• ORDER_COUNT represents total orders per customer across all records

• Preconditions:

• Joined DataFrame contains CID and OID columns

• CID values are valid and not NULL

• Postconditions:

• ORDER_COUNT column exists in DataFrame

• ORDER_COUNT is a positive integer >= 1

• All records for the same CID have identical ORDER_COUNT value

• Error / Validation Conditions:

• If ORDER_COUNT calculation fails, log error and terminate job

• ORDER_COUNT must be >= 1 (since inner join ensures at least one order per customer)

•

FR-TRANSFORM-004: Add Processing Timestamp

• Title: Add ETL Processing Date

• Description:

The system shall add a timestamp column PROCESSED_DATE containing the current date and time when the ETL job is executed.

• Business Objective:

Enable audit trail and data lineage tracking by recording when each record was processed.

• Inputs:

• Enhanced DataFrame from FR-TRANSFORM-003

• Processing Rules / Functional Steps:

1. Capture current timestamp at job execution time
2. Add new column PROCESSED_DATE with timestamp data type
3. Apply same timestamp value to all records in the DataFrame
4. Use format: YYYY-MM-DD HH:MM:SS

• Outputs:

• Final transformed DataFrame with additional column PROCESSED_DATE (Timestamp)

• Preconditions:

• DataFrame from FR-TRANSFORM-003 is available

• System clock is accurate and synchronized

• Postconditions:

• PROCESSED_DATE column exists in DataFrame

• All records have identical PROCESSED_DATE value

• PROCESSED_DATE represents actual job execution time

• Error / Validation Conditions:

• If timestamp generation fails, log error and terminate job

• PROCESSED_DATE must not be NULL

•

FR-LOAD-001: Write Output to S3 in Parquet Format

• Title: Persist Transformed Data to Target Location

• Description:

The system shall write the final transformed dataset to S3 in Parquet format with overwrite mode.

• Business Objective:

Make enriched customer-order data available for downstream analytics and reporting systems.

• Inputs:

• Final transformed DataFrame from FR-TRANSFORM-004

• Columns: CID, FULL_NAME, OID, ONAME, ORDER_COUNT, PROCESSED_DATE

• Processing Rules / Functional Steps:

1. Write DataFrame to S3 location: s3://adif-sdlc/output/customer_orders/
2. Use Parquet file format

3. Apply overwrite mode (replace existing data)

4. Partition data by PROCESSED_DATE

5. Validate write operation success

6. Log count of records written

7. Compare input record count with output record count to detect data loss

• Outputs:

• Parquet files written to s3://adif-sdlc/output/customer_orders/

• Partitioned directory structure by processing date

• Log entry with write statistics (record count, file count, write duration)

• Preconditions:

• Final DataFrame is complete with all required columns

• Target S3 location is accessible with write permissions

• Target S3 bucket exists

• Postconditions:

• Data is successfully written to target location

• Output record count matches input record count

• Parquet files are readable and valid

• Previous data at target location is overwritten

• Error / Validation Conditions:

• If write operation fails, log error and terminate job

• If record count mismatch detected, log error and terminate job

• If S3 location is not accessible, log error and terminate job

• If disk space or quota exceeded, log error and terminate job

•

FR-ERROR-001: Error Handling and Logging

• Title: Comprehensive Error Handling and Audit Logging

• Description:

The system shall implement comprehensive error handling, logging rejected records, and generating error reports for all validation failures.

• Business Objective:

Enable data quality monitoring, troubleshooting, and continuous improvement of data sources.

• Inputs:

• Invalid records from FR-VALIDATE-001 and FR-VALIDATE-002

• Job execution events and errors

• Processing Rules / Functional Steps:

1. Validation Error Logging:

• Capture all records rejected during validation

• Log rejection reason for each record

• Include original record data in error log

• Write error log to designated S3 location

2. Job Execution Logging:

• Log job start time and end time

• Log record counts at each processing stage

• Log transformation metrics (duplicates removed, records joined, etc.)

• Log any warnings or exceptions encountered

3. Error Report Generation:

• Generate summary error report with:

• Total records processed

• Total records rejected

• Breakdown by rejection reason

• Sample of rejected records

• Write error report to S3 location

4. Notification:

• Send job completion notification with status (success/failure)

• Include summary metrics in notification

• Alert on critical errors or data quality issues

• Outputs:

• Error log file with rejected records and reasons

• Error summary report

• Job execution log

• Completion notification

• Preconditions:

• Logging infrastructure is configured

• Error log S3 location is accessible

• Postconditions:

• All validation failures are logged

• Error report is available for review

• Job status is communicated to stakeholders

• Error / Validation Conditions:

• If logging fails, attempt to write to alternate location

• If notification fails, log error but do not fail job

• Ensure error handling does not cause job to hang or crash

•

FR-CONFIG-001: YAML-Based Configuration Management

• Title: Externalized Configuration via YAML

• Description:

The system shall support externalized configuration of all runtime parameters via a YAML configuration file.

• Business Objective:

Enable flexible deployment across environments without code changes and simplify parameter management.

• Inputs:

• YAML configuration file

• Processing Rules / Functional Steps:

1. Read YAML configuration file at job startup

2. Parse and validate configuration parameters

3. Support the following configurable parameters:

• Source S3 path for customers CSV

• Source S3 path for orders CSV

• Target S3 path for output

• Input file format (CSV)

• Output file format (Parquet)

• Write mode (overwrite/append)

• Job logging level (INFO/DEBUG/ERROR)

• Enable/disable metrics collection flag

4. Apply configuration values to job execution

5. Log configuration values used (excluding sensitive data)

• Outputs:

• Loaded configuration object

• Log entry with configuration summary

• Preconditions:

• YAML configuration file exists and is accessible

• YAML file follows expected schema

• Postconditions:

• All job parameters are loaded from configuration

• Job executes using configured values

• Error / Validation Conditions:

• If YAML file not found, log error and terminate job

• If YAML parsing fails, log error and terminate job

• If required parameters are missing, log error and terminate job

• If parameter values are invalid, log error and terminate job

•

FR-TEST-001: Unit Testing Requirements

• Title: Comprehensive Unit Test Coverage

• Description:

The system shall include a comprehensive unit test suite covering all transformation logic, validation rules, and error handling.

• Business Objective:

Ensure code quality, reliability, and maintainability through automated testing.

• Inputs:

• Sample test data (customers.csv, orders.csv)

• Unit test framework (pytest for Python / ScalaTest for Scala)

• Processing Rules / Functional Steps:

1. Test Data Loading:

• Test successful CSV file reading

• Test handling of missing files

• Test handling of empty files

• Test schema validation

2. Test Transformation Logic:

• Test FULL_NAME concatenation with various inputs

• Test proper case conversion

• Test ORDER_COUNT calculation with different scenarios

• Test PROCESSED_DATE generation

3. Test Join Logic:

• Test inner join with matching records

• Test inner join with no matches

• Test join with multiple orders per customer

• Test referential integrity enforcement

4. Test Validation Rules:

• Test NULL value detection

• Test positive integer validation

• Test empty string detection

• Test duplicate removal

5. Test Error Handling:

• Test error logging functionality

• Test continuation after validation failures

• Test job termination on critical errors

• Outputs:

• Unit test suite with 100% test pass rate

• Test coverage report

• Test execution log

• Preconditions:

• Test framework is configured

• Sample test data is available

• Test environment is set up

• Postconditions:

• All unit tests pass successfully

• Test coverage meets minimum threshold

• Test results are documented

• Error / Validation Conditions:

• If any test fails, report failure details

• If test coverage is below threshold, report warning

• Ensure tests are repeatable and deterministic

•

FR-TEST-002: Integration Testing Requirements

• Title: End-to-End Integration Testing

• Description:

The system shall support end-to-end integration testing with sample data to validate complete job execution flow.

• Business Objective:

Verify that all components work together correctly and produce expected results.

• Inputs:

• Sample customers.csv with 8-10 records including edge cases

• Sample orders.csv with 10-15 records including edge cases

• Expected output dataset for comparison

• Processing Rules / Functional Steps:

1. Test End-to-End Execution:

• Execute complete job with sample data

• Validate output is generated successfully

• Compare output with expected results

2. Test Output Schema:

• Verify all required columns exist

• Verify data types match specification

• Verify column order matches specification

3. Test Referential Integrity:

• Verify all orders have valid customer references

• Verify no orphan records in output

4. Test Edge Cases:

• Test with customers having no orders (should be excluded)

• Test with duplicate customer records (should be removed)
• Test with duplicate order records (should be removed)
• Test with NULL values (should be rejected and logged)
• Test with invalid data types (should be rejected and logged)

5. Test Data Accuracy:
• Verify FULL_NAME is correctly derived
• Verify ORDER_COUNT is accurate
• Verify PROCESSED_DATE is populated
• Verify record counts match expectations

• Outputs:
• Integration test results report
• Output dataset for validation
• Test execution log

• Preconditions:
• Sample test data files are prepared
• Expected output dataset is defined
• Test environment mirrors production configuration

• Postconditions:
• All integration tests pass successfully
• Output matches expected results
• Edge cases are handled correctly

• Error / Validation Conditions:
• If output schema does not match, report failure
• If record counts do not match expectations, report failure
• If data accuracy checks fail, report failure
• If edge cases are not handled correctly, report failure

•

3. NON-FUNCTIONAL CONSIDERATIONS

3.1 Performance Requirements
• Scalability: System must process up to 10 million customer records and 50 million order records
• Execution Time: Complete job execution within 30 minutes
• Optimization: Use appropriate partitioning strategies and caching to optimize performance

• Resource Utilization: Efficiently use Glue DPU resources

3.2 Reliability Requirements

• Data Integrity: No data loss during processing

• Consistency: Output data must be consistent with input data after transformations

• Recoverability: Job must be re-runnable without side effects (idempotent)

3.3 Maintainability Requirements

• Code Quality: Production-ready code with proper error handling

• Documentation: Comprehensive README with setup and execution instructions

• Project Structure: Follow best practices for project organization

• Configuration Management: All parameters externalized via YAML

3.4 Testability Requirements

• Test Coverage: Comprehensive unit and integration test coverage

• Test Automation: Automated test execution via testing framework

• Test Data: Reusable sample test datasets with edge cases

•

4. REQUIREMENT TRACEABILITY NOTES

4.1 BRD to FRD Mapping

D Section	FRD Requirement(s)	Notes
--------	-------------------	-------
Customer Data	FR-EXTRACT-001	Direct mapping of source location and sch
Order Data	FR-EXTRACT-002	Direct mapping of source location and sch
Data Integration Rules	FR-TRANSFORM-002	Inner join implementation
Data Quality Rules	FR-VALIDATE-001, FR-VALIDATE-002	Validation rules split by entity
Transformation Rules	FR-TRANSFORM-001, FR-TRANSFORM-003, FR-TRANSFORM-004	Each transformation as separate requireme
xpected Output	FR-LOAD-001	Output specification and write operation
ETL Steps	All FR-EXTRACT, FR-TRANSFORM, FR-LOAD	Complete ETL flow coverage
Error Handling	FR-ERROR-001	Comprehensive error handling
Performance Requirements	Section 3.1	Non-functional requirements
Configuration Parameters	FR-CONFIG-001	YAML-based configuration
Testing Requirements	FR-TEST-001, FR-TEST-002	Unit and integration testing

4.2 Requirement Dependencies

```
```

FR-EXTRACT-001 (Load Customers)

↓

FR-VALIDATE-001 (Validate Customers)

↓

FR-TRANSFORM-001 (Create FULL_NAME)

↓

FR-TRANSFORM-002 (Join Customer-Order) ← FR-VALIDATE-002 (Validate Orders) ← FR-EXTRACT-002 (Load Orders)

↓

FR-TRANSFORM-003 (Calculate ORDER_COUNT)

↓

FR-TRANSFORM-004 (Add PROCESSED_DATE)

↓

FR-LOAD-001 (Write Output)

FR-ERROR-001 (Error Handling) - Cross-cutting concern applied throughout

FR-CONFIG-001 (Configuration) - Applied at job initialization

FR-TEST-001, FR-TEST-002 (Testing) - Applied during development and deployment

```

### 4.3 Completeness Check

All functional requirements are complete and implementation-ready:

- ✓ Source systems and exact file paths specified
- ✓ Input schemas with data types documented
- ✓ Processing rules clearly defined
- ✓ Output specifications complete
- ✓ Error conditions identified
- ✓ Preconditions and postconditions stated
- ✓ Testing requirements defined
-

- END OF FUNCTIONAL REQUIREMENTS DOCUMENT
```